

Project I: Manual Ordinary Differential Equation & Integral Solvers

ALEXANDRA VONHAVEN

Syracuse University

I. PHYSICAL PROBLEMS

I.1. ODE Problem: Coupled Oscillator

The problem I've chosen to model with the ODE solver is the coupled harmonic oscillator. Refer to figure [INSERT FIGURE HERE]. For my particular set-up, each of the two block have masses of m_1 and m_2 . The three springs will all have the same spring constant, k . For mathematical convenience, I assume initial displacements of 1 [m] and initial velocities of 1 $\left[\frac{\text{m}}{\text{s}}\right]$.

The equations of motion for each mass takes the form:

$$m_i \ddot{x}_i = -kx_i + k(x_j - x_i) \quad (1)$$

$$= -2kx_i + kx_j \quad (2)$$

I.1.1 Analytical Solution

We can assume the displacement of each mass will take the standard form for the harmonic oscillator:

$$x_i = A_i \cos(\omega t)$$

We can plug this into the differential equation to get:

$$m_i A_i \omega^2 \cos(\omega t) = -2k A_i \cos(\omega t) + k A_j \cos(\omega t) \quad (3)$$

$$(4)$$

We can plug in the indices 1 and 2, set them equal to each other, and re-write this equation in the form of a matrix:

$$m_1 A_1 \omega^2 \cos(\omega t) = 2k A_1 \cos(\omega t) - k A_2 \cos(\omega t)$$

$$m_2 A_2 \omega^2 \cos(\omega t) = 2k A_2 \cos(\omega t) - k A_1 \cos(\omega t)$$

$$\begin{bmatrix} m_1 \omega^2 - 2k & k \\ k & m_2 \omega^2 - 2k \end{bmatrix} \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

To have a non-trivial value of $\begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$, we need the matrix operating on it to be **singular**, meaning its determinate is zero:

$$\begin{aligned} \begin{vmatrix} m_1 \omega^2 - 2k & k \\ k & m_2 \omega^2 - 2k \end{vmatrix} &= 0 \\ &= (m_1 \omega^2 - 2k)(m_2 \omega^2 - 2k) - k^2 \\ &= m_1 m_2 \omega^4 - 2k(m_1 + m_2) \omega^2 + 3k^2 \\ \omega^2 &= \frac{k}{m_1 m_2} \left[(m_1 + m_2) \pm \sqrt{m_1^2 - m_1 m_2 + m_2^2} \right] \end{aligned}$$

We will call the two frequencies ω_+ and ω_- and define them as:

$$\omega_p^2 = \frac{k}{m_1 m_2} \left[(m_1 + m_2) + \sqrt{m_1^2 - m_1 m_2 + m_2^2} \right]$$

$$\omega_n^2 = \frac{k}{m_1 m_2} \left[(m_1 + m_2) - \sqrt{m_1^2 - m_1 m_2 + m_2^2} \right]$$

We can now plug each ω back into the matrix and solve for the ratio of the amplitudes, A_1 and A_2 :

$$\begin{bmatrix} m_1 \omega_{\pm}^2 - 2k & k \\ k & m_2 \omega_{\pm}^2 - 2k \end{bmatrix} \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} = 0$$

$$\implies \frac{A_2}{A_1} = 2 - \frac{m_1 \omega_{\pm}^2}{k}$$

Now, we can use our general solution from earlier to look for a particular solution. Since we know the oscillators' displacements can be written as a sum of the normal mode frequencies, $\omega_{\pm}$. For each ω , we will add a sinusoidal term to account for the initial conditions.

Returning to general index notation, we can express the general solutions as:

$$x_i = \sum_n C_n \cos(\omega_n t) + S_n \sin(\omega_n t)$$

where C_n and S_n are the coefficients of the cosine and sine terms respectively. We can solve for the coefficients by getting the general solutions for $t = 0$:

$$\begin{aligned} x_i(t=0) &= A_i \sum_n \omega_n (C_n \cos(\omega_n \cdot 0) + \cancel{S_n \sin(\omega_n \cdot 0)}) \\ &= A_i \sum_n \omega_n C_n \end{aligned}$$

Since we see that the sinusoidal terms cancel at $t = 0$, it follows that we can use the initial values of $\dot{x}_i$ to get the remaining coefficients:

$$\begin{aligned} \dot{x}_i(t=0) &= A_i \sum_n \omega_n (\cancel{C_n \cos(\omega_n \cdot 0)} + S_n \sin(\omega_n \cdot 0)) \\ &= A_i \sum_n \omega_n D_n \end{aligned}$$

For the the particular indices 1 and 2, I use the fact that the amplitudes can be related by $A_1 \left(\frac{A_2}{A_1} \right) = A_2$

$$x_1 = \sum_n (C_n \cos \omega_n t + S_n \sin \omega_n t)$$

$$x_2 = \sum_n \left(\frac{A_2}{A_1} \right)_n (C_n \cos \omega_n t + S_n \sin \omega_n t)$$

Finally, I substitute the amplitude ratio with the variable r to get the form:

$$x_1 = C_1 \cos \omega_p t + S_1 \sin \omega_p t + C_2 \cos \omega_n t + S_2 \sin \omega_n t$$

$$x_2 = r_p (C_1 \cos \omega_p t + S_1 \sin \omega_p t) + r_n (C_2 \cos \omega_n t + S_2 \sin \omega_n t)$$

for the exact analytical solutions.

I.2. Integration Problem: Quantum Particle in a Box of Infinite Depth

The problem I've chosen to solve with my numerical integrator is a system where a particle of mass m is placed in a potential "box" of width a that varies with its position, x , as:

$$V(x) = \begin{cases} 0, & |x| \leq \frac{a}{2} \\ \infty & \text{otherwise} \end{cases}$$

The particle is completely free to move within the infinitely steep "walls" of the potential.

I.2.1 Analytic Solution

To get an exact solution for the particle's position we use the time-independent Schrödinger Equation:

$$-\frac{\hbar^2}{2m} \frac{d^2\psi}{dx^2} = E\psi$$

Where E is the total energy of the particle and ψ is its wave function at $t = 0$. We can put this differential equation into a similar form as the ODE problem from the previous sub-section:

$$\frac{d^2\psi}{dx^2} = -\frac{2mE}{\hbar^2}\psi$$

From our previous results, we know we can express the solution as a sum of oscillatory terms with a frequency, which will be called ω for self-consistency within the report even if that's not the standard convention, defined as the square root of the absolute value of the coefficient on the right:

$$\psi(x) = A \sin(\omega x) + B \cos(\omega x)$$

The initial conditions for this system are a bit less straightforward. Rather than just solving for $x = 0$, we need to invoke the condition that the wavefunction, ψ , is **continuous** at the potential walls, meaning $\psi(-\frac{a}{2}) = \psi(\frac{a}{2}) = 0$. We can perform a sort of "cheat" where we translate the origin of our coordinates a factor of $\frac{a}{2}$ to the right. Hence, the leftmost wall is no longer at $x = -\frac{a}{2}$, but instead at $x = 0$. Now, we actually *can* plug in $x = 0$:

$$\psi(0) = A \sin(\omega \cdot 0) + B \cos(\omega \cdot 0) = 0$$

Now, for the rightmost boundary:

$$\psi(a) = A \sin(\omega a) = 0$$

To get a non-trivial solution, we require that ωa needs to be an integer multiple of π . Hence, we can write the acceptable frequencies as:

$$\omega_n = \frac{n\pi}{a}, n \in \mathbb{Z}^+$$

Now we can write:

$$\psi_n(x) = A \sin\left(\frac{n\pi}{a}x\right)$$

Plugging this back into the time-independent Schrödinger Equation, we get the following expression for the discrete energy values, E_n :

$$\begin{aligned} -\left(\frac{n\pi}{a}\right)^2 A \sin\left(\frac{n\pi}{a}x\right) &= -\frac{2mE_n}{\hbar^2} A \sin\left(\frac{n\pi}{a}x\right) \\ \implies E_n &= \boxed{\frac{(n\pi\hbar)^2}{2ma^2}} \end{aligned}$$

Since all quantum mechanical wavefunctions must be normalized, we end up with the final, particular solution:

$$\boxed{\psi(x) = \sqrt{\frac{2}{a}} \sin\left(\frac{n\pi}{a}x\right)}$$

II. NUMERICAL METHODS

II.1. Symplectic Euler & 4th-Order Runge-Kutta ODE Solvers

II.1.1 Overview

The symplectic Euler and fourth-order Runge-Kutta methods are both methods used to numerically approximate differential equations. Each method is derived from the same general principle. Let's begin with the symplectic Euler method.

Suppose we have a second-order differential equation where we know the value of $\ddot{y}(y)$ (ie: we have the second derivative of some function $y(t)$ with respect to y itself). The symplectic (or semi-implicit) Euler method

The standard Runge-Kutta 4 approximation method is defined as:

$$\begin{aligned} k_1 &= \Delta x f(x_0, y_0) \\ k_2 &= \Delta x f(x_0 + \frac{\Delta x}{2}, y_0 + \frac{k_1}{2}) \\ k_3 &= \Delta x f(x_0 + \frac{\Delta x}{2}, y_0 + \frac{k_2}{2}) \\ k_4 &= \Delta x f(x_0 + \Delta x, y_0 + k_3) \\ y_{n+1} &= y_n + \frac{\Delta x}{6} (k_1 + 2k_2 + 2k_3 + k_4) \end{aligned}$$

We can generalize this to the form of the solution to the coupled oscillator for each mass, m_i :

$$f(x_i, x_j, m_i) = \frac{1}{m_i} (-2k x_i + k x_j), \quad (5)$$

II.1.2 Python Implementation

When modeling the coupled oscillator system using both numerical ODE solvers, I use a constant mass ratio is modeled as $\frac{m_2}{m_1} = 10$. The one exception to this was a limiting scenario that will be discussed in the validation section.

The symplectic Euler and fourth-order Runge-Kutta methods were each implemented through iterative python functions. Since it's only accurate to the second order in Δt , the Euler solver is able to calculate the new state for each mass without needing information about the other.

The Runge-Kutta solver, however, requires the current positions and velocities of both masses in order to calculate a new state. In fact, the coupled oscillator invokes dependence on separate values for each k_i ($i \in \{1, 2, 3, 4\}$) for each mass's position and velocity.

II.1.3 Truncation Errors

Recall the general formula for the Taylor Approximation of some function $y(x)$ around some initial x_0 :

$$\begin{aligned} y(x) &= \sum_{n=1} y^{(n)}(x_0) \frac{(x - x_0)^n}{n!} \\ &\approx y(x_0) + y'(x_0)(x - x_0) + y''(x_0) \frac{(x - x_0)^2}{2} + \dots \end{aligned}$$

Since our approximation takes the form:

$$v_i^{n+1} = v_i^n + f(x_i^n, x_j^n, t) \Delta t$$

Since $f(x_i^n, x_j^n, t)$ is the acceleration of each mass, it follows that the analytical solution can be obtained through the approximated solution through:

$$x_i^{n+1}(\text{exact}) = x_i^n + v_i^n \Delta t + f(x_i^n, x_j^n, t) \Delta t^2 + \mathcal{O}(\Delta t^3)$$

Where $\mathcal{O}(\Delta t^3)$ corresponds to the terms of the Taylor expansion of the third order and higher. Thus, the model should have a **local truncation error** of $\mathcal{O}(\Delta t^3)$. Due to the compounding of the local error over the duration of the time-interval, however, the **global truncation error**, is about an order of approximation higher than the local error, in our case, $\mathcal{O}(\Delta t^2)$.

For the general solution for y_{n+1} using the fourth-order Runge-Kutta method, we take the sum of five terms up to the fourth-order correction of $f(x, y)$. Hence, using the same rationale to determine the truncation errors for the Euler method, we get a **local truncation error** of $\mathcal{O}(\Delta x^5)$ and a **global truncation error** of $\mathcal{O}(\Delta x^4)$.

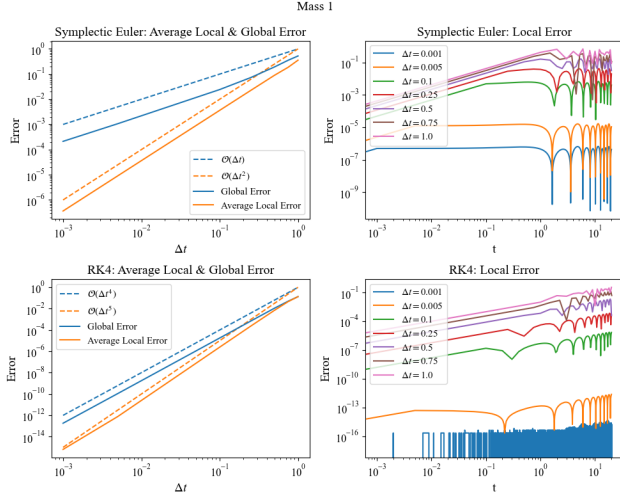


Figure 1: Truncation errors for mass 1

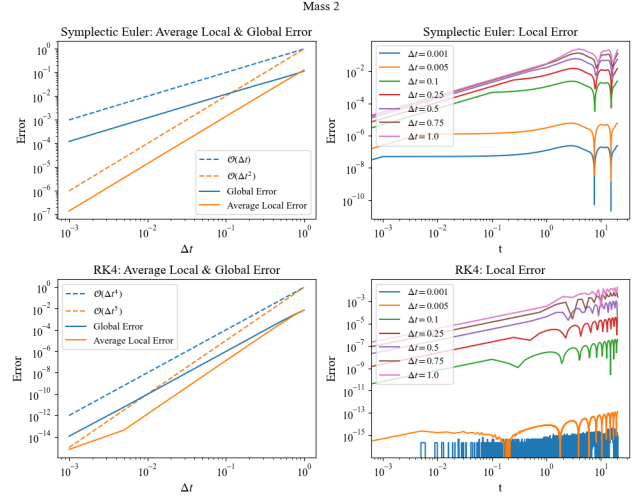


Figure 2: Truncation errors for mass 2

II.2. Reimann Integrator

II.2.1 Overview

The formula for a Reimann sum is:

$$\int_1^n f(x(t))dx = \lim_{\Delta t \rightarrow 0} \sum_{i=1}^n f(x_i)\Delta t_i$$

for a constant step-size Δx . I will add more after 5 PM.

II.2.2 Python Implementation

The python function for the Reimann integrator is extremely simple. I wrote two functions for it. The first simply calculates the area of a rectangle with a height of $f(x)$ and a width of Δx . The second is a recursive algorithm that takes the area at some value "A" that changes by Δx until it reaches the base case where A hits a maximum value specified by the user.

II.2.3 Truncation Errors

The fundamental theorem of calculus tells us that, for some function $f(x)$:

$$f(x) = \frac{d}{dx} \left[\int_{x_0}^x f(t)dt \right]$$

To get the local truncation error, we can perform a Taylor Expansion of the integral itself:

$$\int_{x_i}^{x_i+\Delta x} f(x) dx = \int_{x_i}^{x_i+\Delta x} [f(x_i) + f'(x_i)(x - x_i) + \dots] dx \quad (6)$$

$$= f(x_i) \Delta x + \frac{\Delta x^2}{2} f'(x_i) + \dots \quad (7)$$

We can see that only the first term, $f(x_i)\Delta x$ is calculated in my left-Reimann integrator. Hence, the method has a **local truncation error** of $\mathcal{O}(\Delta x^2)$.

On the interval x_i to x_f where each step is evenly spaced by Δx , we get a total of $n = \frac{x_f - x_i}{\Delta x}$ steps. Thus, the **global truncation error** accumulated over the entire interval with $n \propto \frac{1}{\Delta x}$ steps is then $\mathcal{O}(\Delta x)$.

Based on this definition of an integral combined with the Reimann approximation formula used, it's

Recall the Taylor expansion form described in the previous sub-section.

III. VALIDATION

III.1. Coupled Oscillator

III.1.1 Comparisons

We can compare the numerical solutions of each ODE solver to the analytic solution derived in the first section. The accuracy of the numerical model most heavily depends on the time-step and the difference between the two masses. I assert that mass 2 is ten times more massive than mass 1.

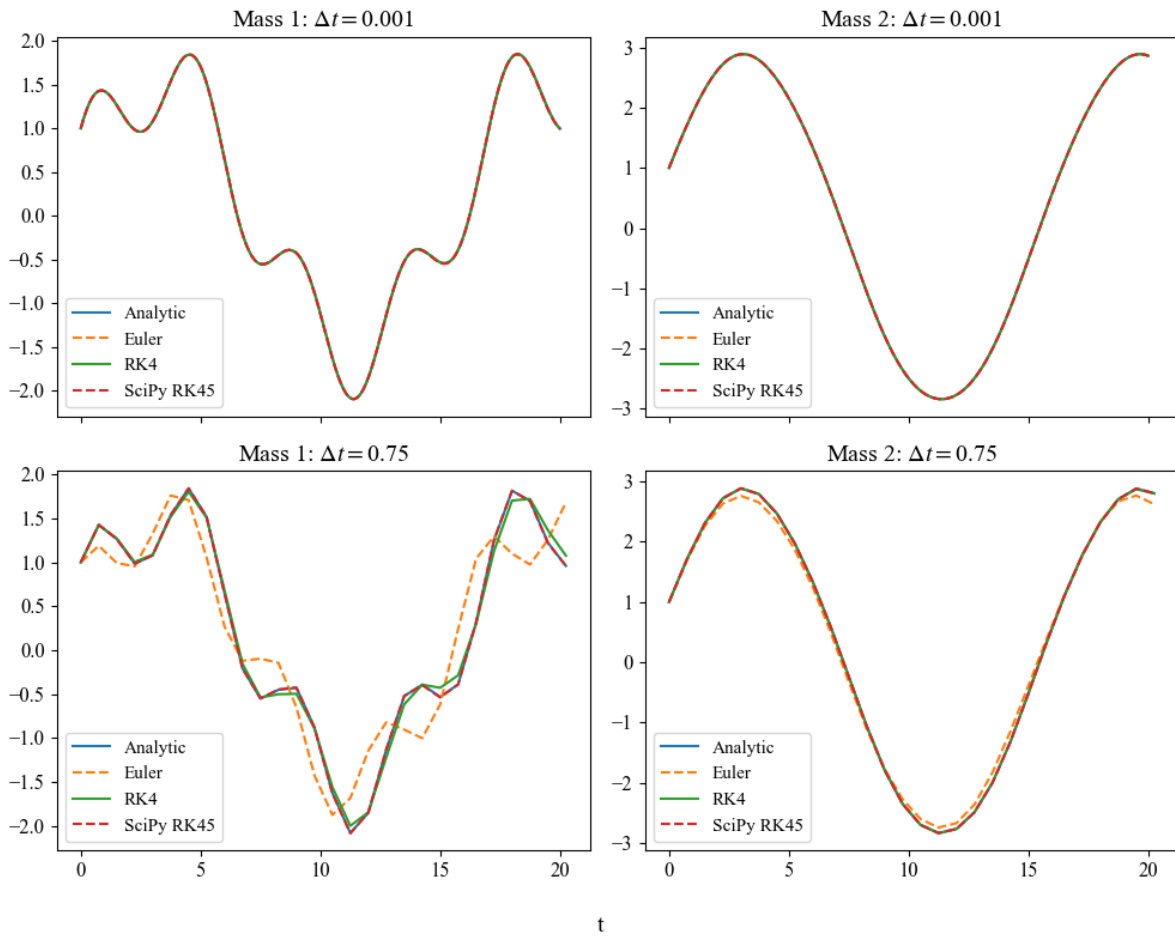


Figure 3: Solutions to the coupled oscillator with $m_2 = 10m_1$ plotted using the SI Euler and RK4 methods written for this project, SciPy `solve_int` RK45 solver, and the real analytic solution at small (0.001) and large (0.75) values of Δt

As shown in figure 3, both algorithms fit the analytic solution almost perfectly for the small Δt . For the case of larger Δt , however, we find that the fits become less accurate. Also notice that this loss of accuracy is significantly more pronounced for the smaller mass. It is also clear that the RK4 algorithm fares much better with regards to its fit to the analytic solution than the Euler algorithm at large Δx .

We can further validate the

III.1.2 Special Property/Limiting Case I: Modeling Energy Conservation

One way we can more thoroughly test these algorithms is by figuring out whether or not they follow energy conservation. The potential, kinetic, and total energies for the coupled oscillator take the form:

$$U = k \left(x_1^2 + x_2^2 - x_1 x_2 \right) \quad (8)$$

$$T = \frac{1}{2} m_1 v_1^2 + \frac{1}{2} m_2 v_2^2 \quad (9)$$

$$E = T + U = \frac{1}{2} m_1 v_1^2 + \frac{1}{2} m_2 v_2^2 + k \left(x_1^2 + x_2^2 - x_1 x_2 \right) \quad (10)$$

Figure ?? shows these three energy equations calculated using each of the algorithms and compares them to the "analytic" solution. Note, as will be explained in the "Errors and shortcomings" section, the analytic values were **NOT** calculated correctly. Due to time restrictions, I was not able to produce a function for the derivative of the analytic solution. As we saw in figure 3, the RK4 algorithm produced a near-perfect fit with the analytic

solution. Thus, I used the velocity values calculated using that algorithm in place of the analytic velocity values.

We can see in Figure 4 that both algorithms produce a value of E that is almost perfectly constant at $\Delta t = 0.001$. However, at $\Delta t = 0.75$, the algorithms become less accurate. The energy produced by the Euler algorithm instead oscillates around a constant energy around the same value seen in the smaller time step. This plot is where we can really see the flaws in using the RK4 velocity, as the "analytic" solution gradually begins to oscillate as well at higher t values. Notice that the RK4 algorithm produces a straight line with a slight linear decrease. This suggests that the algorithm will gradually underestimate the energy by a greater margin as time goes on.

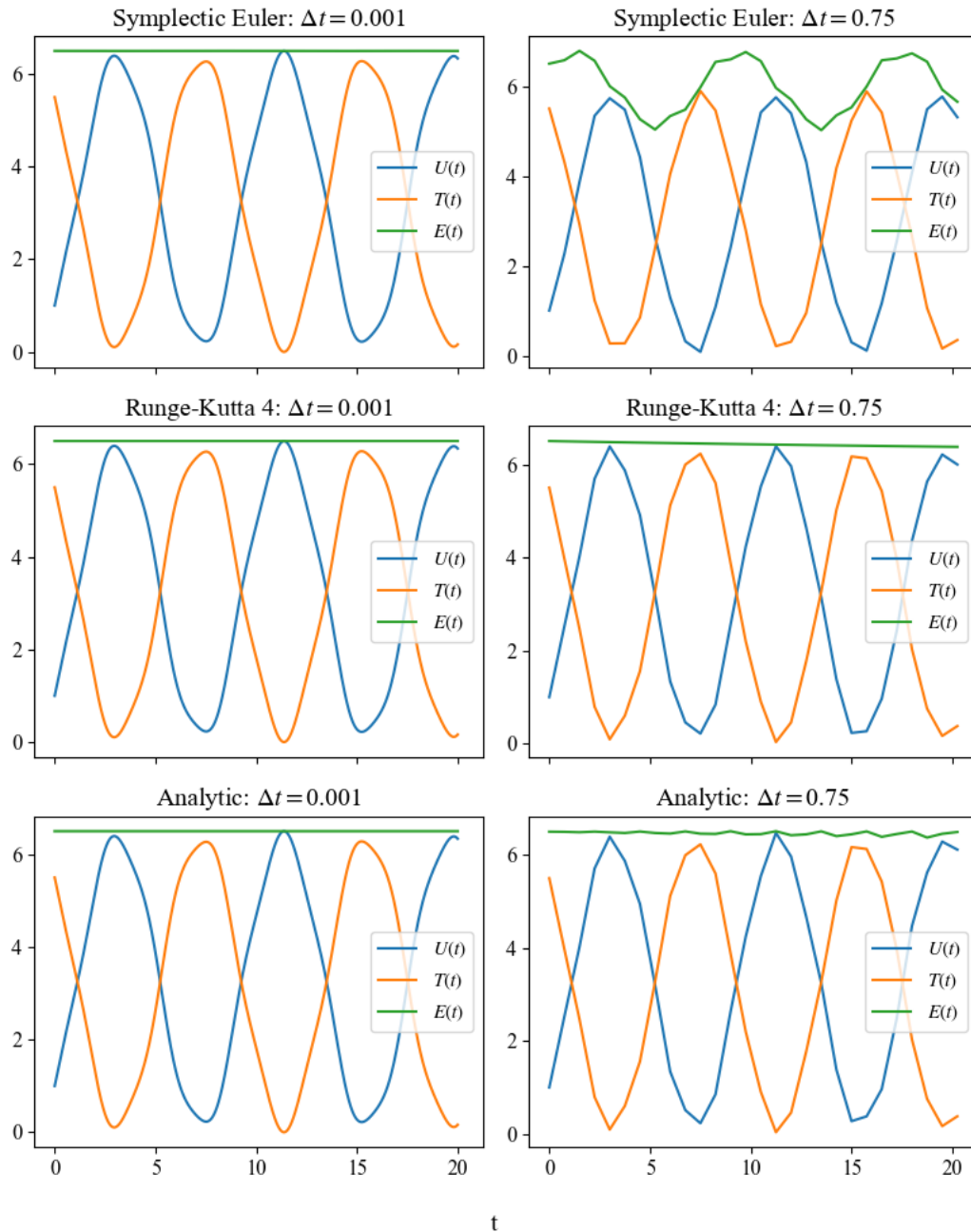


Figure 4: Testing whether energy is conserved by the SI Euler and RK4 methods at $\Delta t = 0.001$ and $\Delta t = 0.75$

III.1.3 Special Property/Limiting Case II: $m_1 = m_2$ Case

In the case where $m_1 = m_2$, the solutions for each mass should converge to the same value. If the Euler and RK4 algorithms work as intended, they should each produce identical curves for each mass. As would be expected, these algorithms produce accurate results for small Δt . We can once again see, however, that the RK4 algorithm

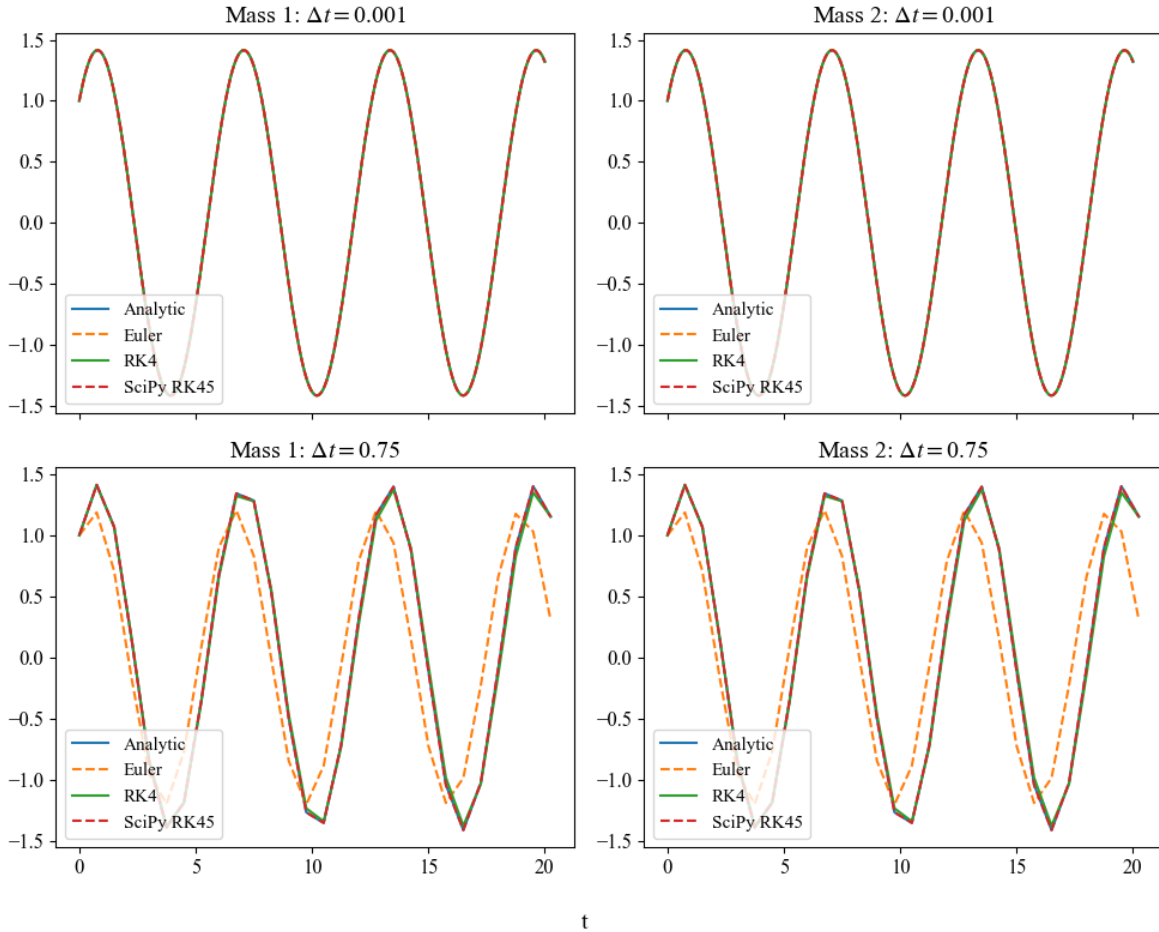


Figure 5: Same quantities plotted in 3 in the case where $m_1 = m_2$

preserves its fit with the analytic solution much better than the Euler algorithm. Also note that my RK4 method overlays nearly perfectly with the solution produced by the SciPy solver. Despite inaccuracies, you can tell from 5 that the mass curves, even for the Euler model, are consistently equivalent for both of the masses. Note that there was a figure created overlaying the two mass curves that I do not have time to reproduce here.

III.1.4 Errors & Shortcomings

The most significant shortcoming of the simulations is that the local truncation errors were not properly computed. Due to underestimating the time commitment required for this project, I ended up not having time to properly compute the analytic solution for the velocity. Due to the visual fit demonstrated in figure 4, I opted to use the velocity array I got from the RK4 function. While, as described in the previous section where I derived the global and local truncation errors, the RK4 method is fairly precise, it is not perfect and I recognize that assuming the model is sufficiently precise does somewhat defeat the purpose of testing it.

Another shortcoming of the analysis is how the expected errors were calculated

Let's start with calculating what the theoretical truncation errors should

III.2. Particle in Infinite Box

III.2.1 Comparisons

We can compare the result of the Riemann integrator by plotting it over the analytic solution derived in the first section, as is shown in Figure 6. We can see that the Riemann integrator produced a near perfect fit to the

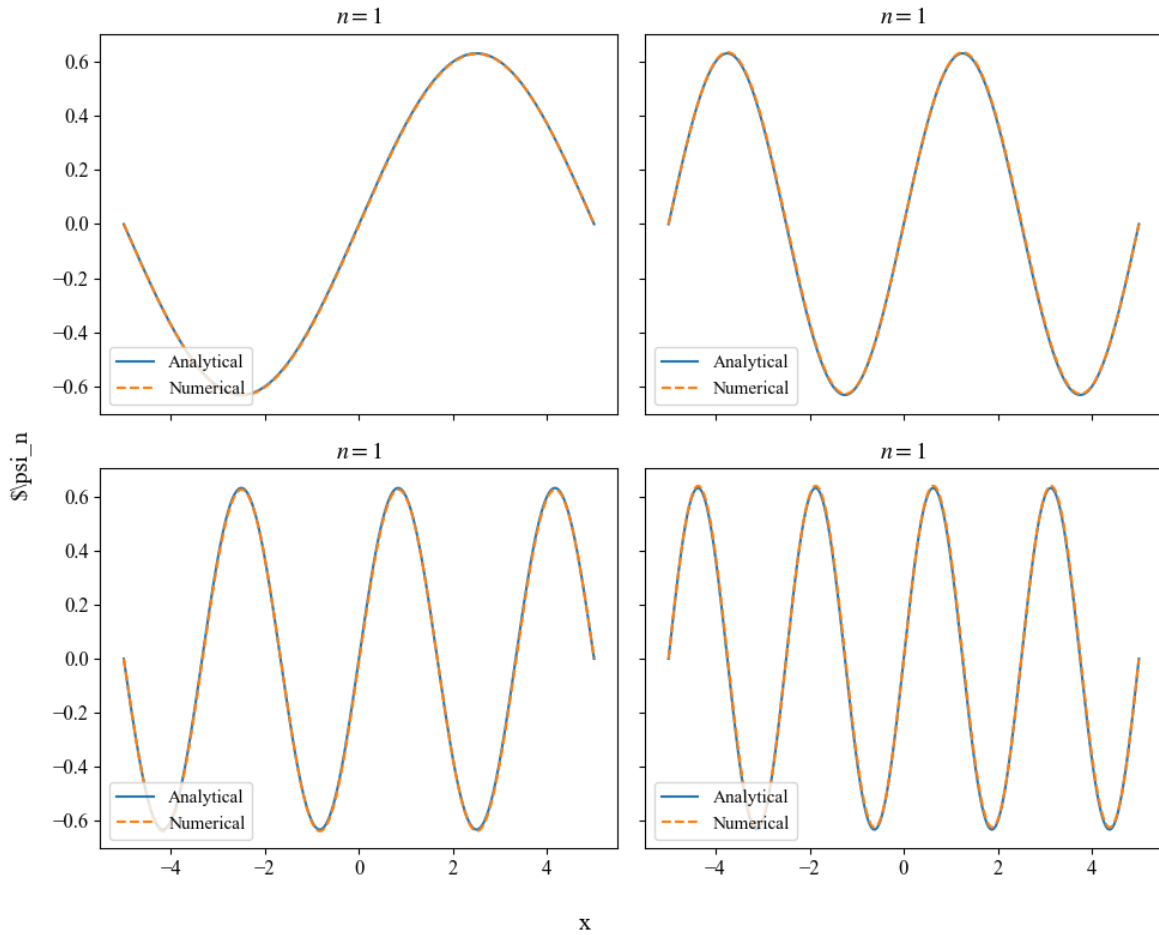


Figure 6: Comparison of the numerical and analytic solution for the ground state and first three excited states of the infinite square well at an interval $\Delta x = 0.01$. The Riemann integrator appears to produce a good fit for each energy level. Note that I was out of time when I saw that the y-axis label, which should read ψ_n , was not properly formatted.

analytical wavefunction at the small interval of $\Delta x = 0.01$. Note that, once again due to time constraints (I really did underestimate how long this project would take), I do not have a plot showing a larger Δx , however this is taken into account in the succeeding subsection regarding truncation errors.

We can further test the validity of the code through testing various identities of the wavefunction. Both of the cases I examine came directly from exercises in ?].

III.2.2 Special Property/Limiting Case I: Convergence of $\langle x^2 \rangle$ at High n

The following was based on exercise 2.4 in ?] The expectation value of the position, x , squared is:

$$\langle x^2 \rangle_n = \frac{a^2}{3} - \frac{a^2}{2n^2\pi^2} \quad (11)$$

As this approaches higher energy levels, n , we expect the second term to disappear. Thus, this expectation value should converge to $\frac{a^2}{3}$, where a is the width of the well.

$$\lim_{n \rightarrow \infty} \langle x^2 \rangle_n = \frac{a^2}{3} \quad (12)$$

We can see from Figure 7, which shows the expectation value for the first 24 excited states, that this appears to hold true.

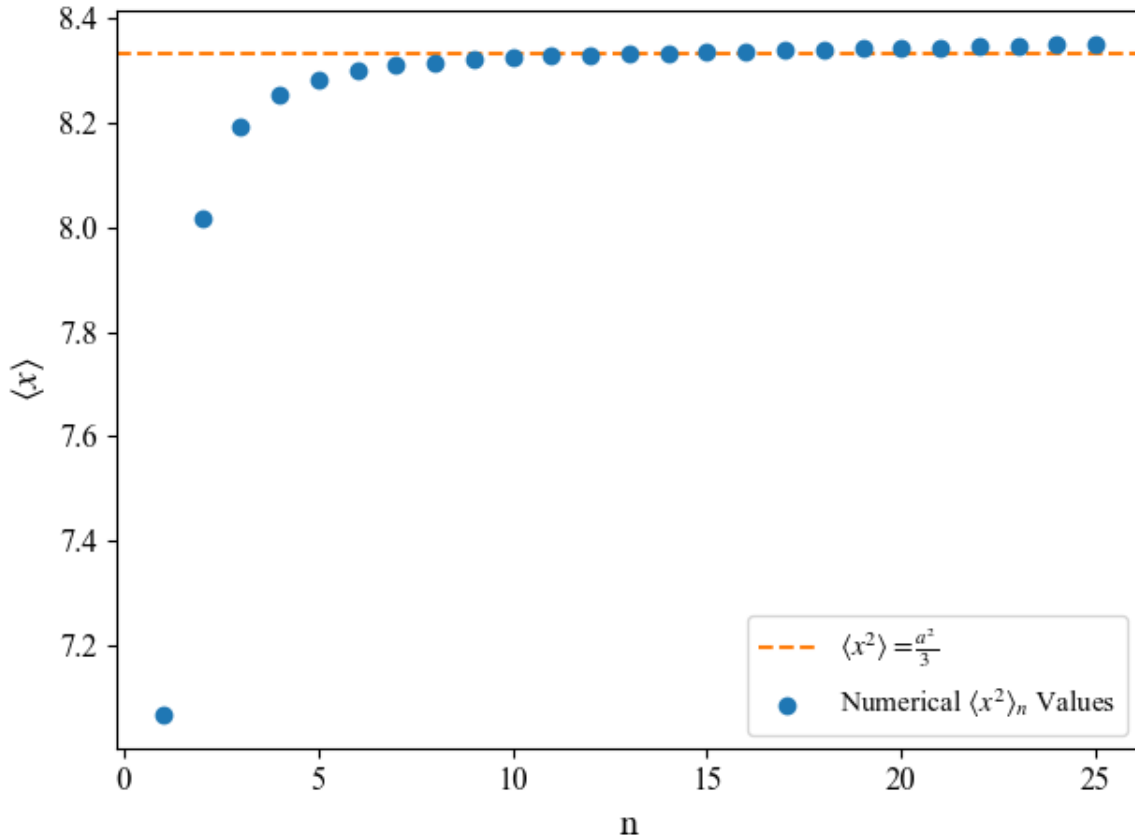


Figure 7: Figure showing $\langle x \rangle$ as a function of the energy level, n . The ground state has been excluded due to scaling. We can see from the figure that $\langle x \rangle$ converges to $\frac{a^2}{3}$

III.2.3 Special Property/Limiting Case II: Property of Nodes

Next, we test a property asserted by [?] Problem 2.45:

$$\psi'_{n_1}(x_2) \psi_{n_2}(x_2) - \psi'_{n_1}(x_1) \psi_{n_2}(x_1) = \frac{2m}{\hbar^2} (E_{n_2} - E_{n_1}) \int_{x_1}^{x_2} \psi_{n_1}(x) \psi_{n_2}(x) dx \quad (13)$$

III.2.4 Errors & Shortcomings

One shortcoming of the Riemann integrator I implemented is that its recursive nature limits the step size. Most Python interpreters have a default recursion depth of 1000. This means that, over a displacement of 20 units, the step size can't be any smaller than 0.01 units. Considering the Riemann integrator has a global truncation error on the order of $\mathcal{O}(\Delta x)$, this can create more significant error than it would for, say, the numerical ODE solvers that are accurate to higher orders.

III.3. Truncation Errors

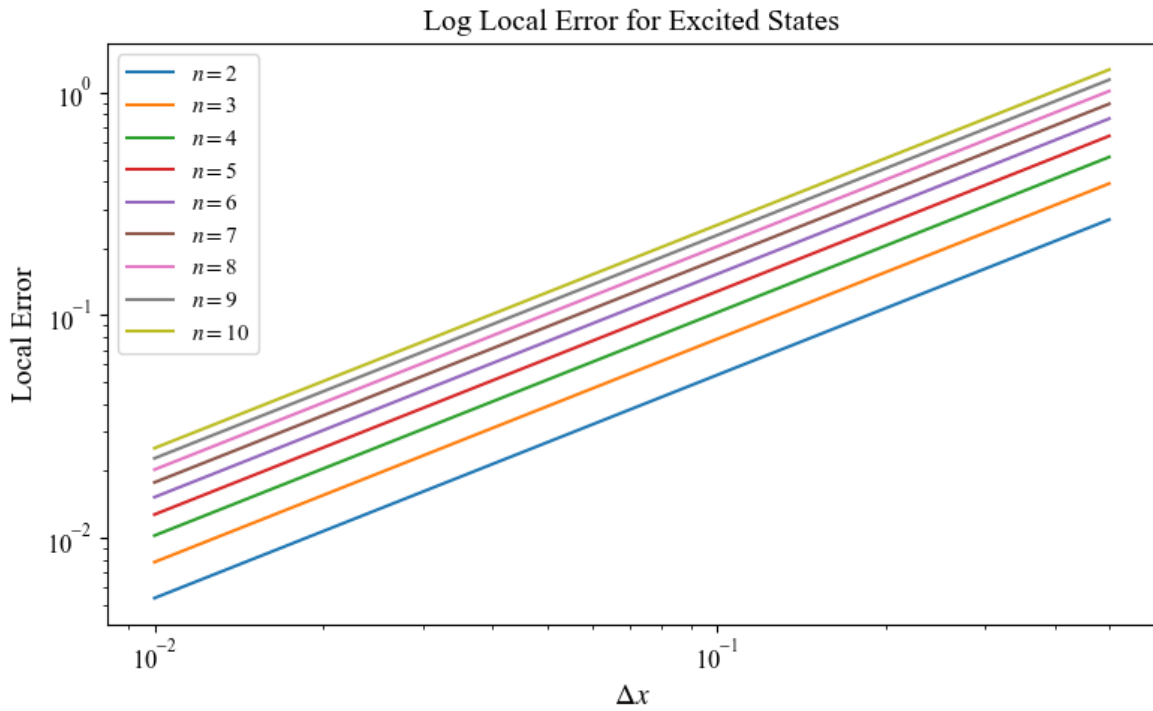


Figure 8: Local error as a function of Δx for the ground state of the infinite square well. Note that this follows the expectation of a truncation error on the order of $\mathcal{O}(\Delta x^2)$

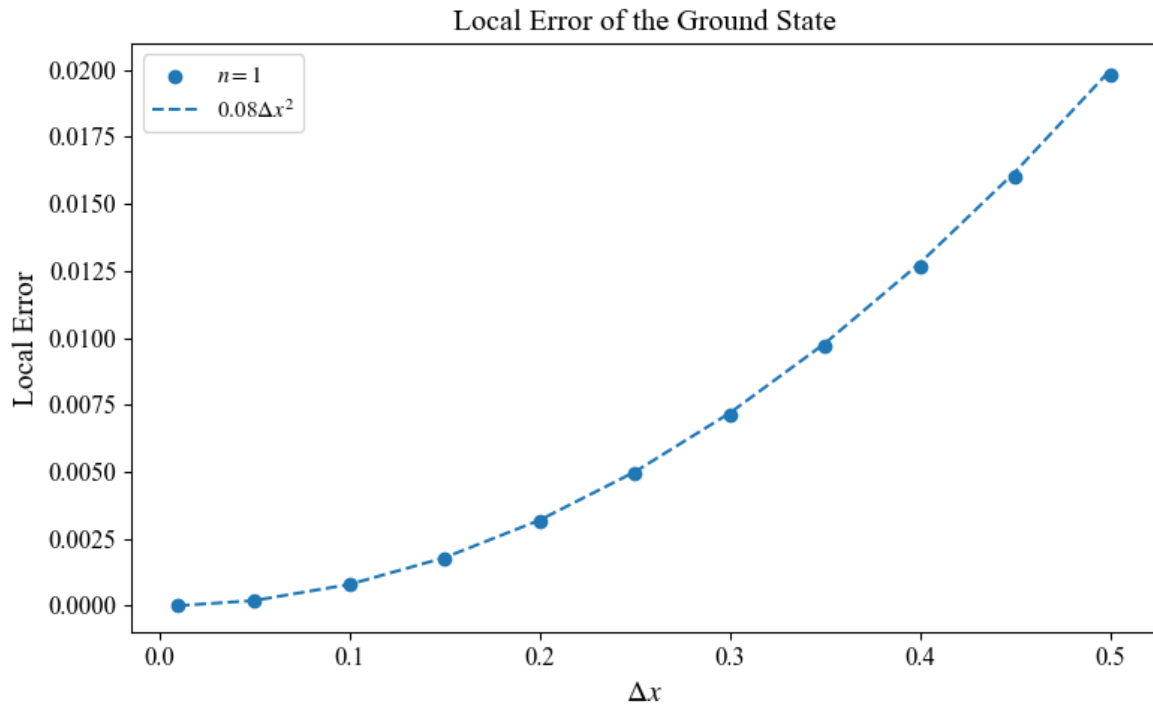


Figure 9: Logarithmic plots of the local error values for the excited states of the infinite square well. Note how each state has the same slope and that the lines begin to converge at higher n .

Global truncation error will be added after 5 PM.

n_1	n_2	Nodes (x_1, x_2)	LHS - RHS
3	2	$\frac{a}{3}, \frac{2a}{3}$	
5	6	$\frac{a}{3}, \frac{2a}{3}$	
8	9	$\frac{a}{3}, \frac{2a}{3}$	

Table 1: Verification of the node-energy identity (Griffiths 2.45) for several (n_1, n_2) pairs.

REFERENCES

IV. ACKNOWLEDGMENTS

I would like to acknowledge David J. Griffiths and Richard Fitzpatrick for their works on quantum mechanics and waves/oscillations respectively. Their works were a frequent resource I turned to as an undergraduate, and they've proven to be just as useful now.

I would also like to acknowledge the copy-paste feature on my MacBook. I would not have gotten through this without copy-paste.